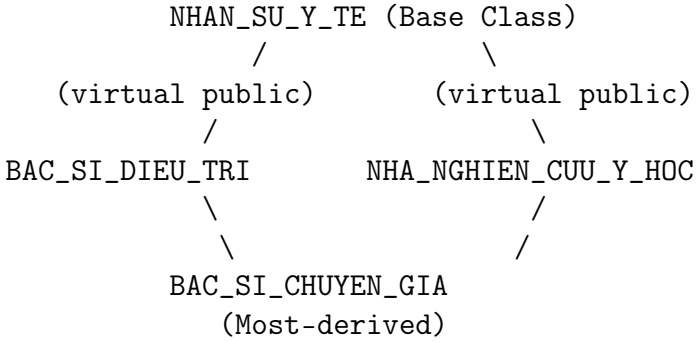
PRACTICE 01

Course: Object-Oriented Programming with C++
Topic: Diamond Inheritance and Dynamic Polymorphism (Virtual Functions)
Time allowed: 75 minutes | **Total:** 10.0 points

Problem Description

An international general hospital needs to develop a management system for Specialist Physicians (`BAC_SI_CHUYEN_GIA`). These professionals both perform surgical diagnosis and treatment and lead clinical medical research projects. To prevent data conflicts and duplicated memory storage for basic personnel information, the system must use the diamond-inheritance pattern with a virtual base class.



Requirements

Question 1 (2.0 points): Implement the top-level base class `NHAN_SU_Y_TE`.

- Protected data members: `ma_so` (`string`), `ho_ten` (`string`), and `he_so_luong` (`double`).
- Provide a default constructor and a parameterized constructor.
- Provide a virtual destructor, `virtual ~NHAN_SU_Y_TE()`, to ensure safe polymorphic memory cleanup.
- Provide the virtual functions `virtual void nhap()`, `virtual void xuat() const`, and either a pure virtual function or a virtual function returning a contribution score: `virtual double tinh_diem_cong_hien() const`.

Question 2 (2.0 points): Implement static members for memory/object tracking.

- Declare `static int so_luong_nhan_su` in `NHAN_SU_Y_TE` to count the objects currently existing in memory.
- Increment the static variable in the constructors and decrement it in the destructor.
- Define the static variable outside the class and implement `static void in_so_luong()` to display the current number of objects.

Question 3 (2.0 points): Overload the comparison operator `operator<`. Overload `bool operator<(const NHAN_SU_Y_TE& other) const` based on the contribution scores returned by `tinh_diem_cong_hien()`. Return `true` if the current object's contribution score is lower than that of the object being compared; otherwise, return `false`.

Question 4 (2.0 points): Design the diamond-inheritance hierarchy.

- **Branch 1:** `BAC_SI_DIEU_TRI` virtually publicly inherits from `NHAN_SU_Y_TE` and adds `so_ca_phau_thuat` (`int`) and `phu_cap_mo` (`double`). Provide `nhap_dt()` and `xuat_dt()`.
- **Branch 2:** `NHA_NGHIEN_CUU_Y_HOC` virtually publicly inherits from `NHAN_SU_Y_TE` and adds `so_bai_bao_isi` (`int`) and `kinh_phi_de_tai` (`double`). Provide `nhap_nc()` and `xuat_nc()`.
- **Converging class:** `BAC_SI_CHUYEN_GIA` inherits from both `BAC_SI_DIEU_TRI` and `NHA_NGHIEN_CUU_Y_HOC`.
- The `BAC_SI_CHUYEN_GIA` constructor must directly call the virtual base class `NHAN_SU_Y_TE` constructor in its member-initializer list.
- Override `nhap()` (calling `NHAN_SU_Y_TE::nhap()` exactly once) and `xuat()`.
- Implement the following contribution-score formula:

Contribution Score = (`so_ca_phau_thuat`$\times$2.0)+(`so_bai_bao_isi`$\times$15.0)+(`he_so_luong`$\times$5.0)

Question 5 (2.0 points): Dynamic allocation, polymorphic sorting, and output in `main()`.

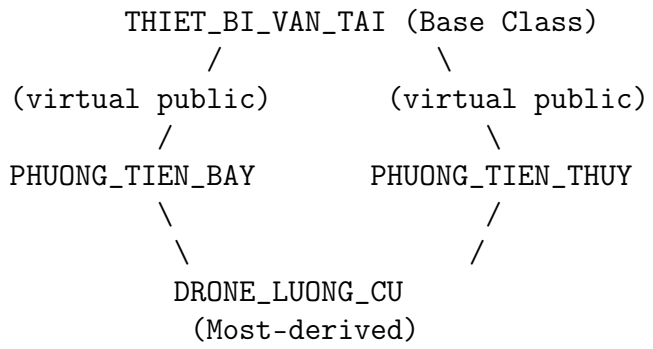
- Call the static function to check the initial object count.
- Dynamically allocate an array of n pointers or an array of n `BAC_SI_CHUYEN_GIA` objects.
- Input the details of n specialist physicians and check the number of currently allocated objects.
- Use `operator<` to sort the list in descending order of contribution score.
- Display the sorted list as a formatted table using `<iomanip>` (for example, `setw` and `left`).
- Release the dynamically allocated memory and verify that the static counter has returned to zero.

PRACTICE 02

Course: Object-Oriented Programming with C++
Topic: Diamond Inheritance and Dynamic Polymorphism (Virtual Functions)
Time allowed: 75 minutes | **Total:** 10.0 points

Problem Description

A defense technology institute is developing a line of Amphibious Unmanned Vehicles (`DRONE_LUONG_CU`) that can operate in the air, like unmanned aerial vehicles, and travel on the water's surface or dive to shallow depths, like seaplanes. The control software must model these devices using diamond inheritance.



Requirements

Question 1 (2.0 points): Implement the top-level base class `THIET_BI_VAN_TAI`.

- Encapsulated data members: `ma_dinh_danh` (`string`), `hang_san_xuat` (`string`), and `trong_luong_kg` (`double`).
- Provide a default constructor, a parameterized constructor, and a virtual destructor (`virtual ~THIET_BI_VAN_TAI()`).
- Provide the virtual methods `virtual void nhap()`, `virtual void xuat() const`, and `virtual double tinh_hieu_suat_nang_luong() const`.

Question 2 (2.0 points): Manage object lifetimes using a static variable.

- Declare `static int tong_so_thiet_bi` to count the number of device objects currently in memory.
- Automatically update `tong_so_thiet_bi` when objects are created or destroyed.
- Define the static variable outside the class and implement `static void hien_thi_tong_thiet_bi(`

Question 3 (2.0 points): Overload the comparison operator `operator>`. Overload `bool operator>(const THIET_BI_VAN_TAI& other) const` to compare energy-efficiency values returned by `tinh_hieu_suat_nang_luong()`. Return `true` if the current device has a higher energy-efficiency score than the device being compared.

Question 4 (2.0 points): Design the multi-branch diamond-inheritance hierarchy.

- **Branch 1:** `PHUONG_TIEN_BAY` virtually publicly inherits from `THIET_BI_VAN_TAI` and adds `do_cao_bay_toi_da_m` (`double`) and `toc_do_gio_kmh` (`double`).
- **Branch 2:** `PHUONG_TIEN_THUY` virtually publicly inherits from `THIET_BI_VAN_TAI` and adds `do_sau_lan_m` (`double`) and `hai_ly_toi_da` (`double`).
- **Converging class:** `DRONE_LUONG_CU` inherits from both branches.
- The `DRONE_LUONG_CU` constructor must directly call the `THIET_BI_VAN_TAI` constructor through its member-initializer list.
- Override the virtual methods `nhap()` and `xuat()`.
- Implement the following efficiency formula:

$$\text{Energy Efficiency} = \frac{(\texttt{do_cao_bay_toi_da_m} \times 0.6) + (\texttt{hai_ly_toi_da} \times 1.8)}{\texttt{trong_luong_kg}}$$

Question 5 (2.0 points): Handle dynamic arrays and sorting in `main()`.

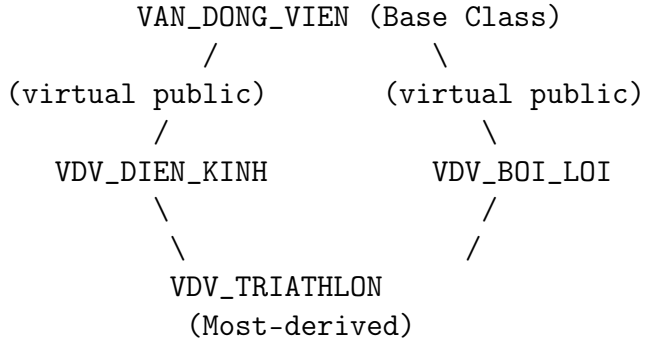
- Initialize and check the static counter.
- Input a list of n `DRONE_LUONG_CU` devices allocated dynamically using `new[]`.
- Use `operator>` to sort the array in ascending order of energy efficiency.
- Display the resulting data in an aligned table using `<iomanip>`.
- Release the dynamic array using `delete[]` and print the number of remaining devices to verify that destruction was successful.

PRACTICE 03

Course: Object-Oriented Programming with C++
Topic: Diamond Inheritance and Dynamic Polymorphism (Virtual Functions)
Time allowed: 75 minutes | **Total:** 10.0 points

Problem Description

The National Sports Training Center needs to manage Triathlon Athletes (`VDV_TRIATHLON`), who compete in swimming, cycling, and running. Each athlete has the physical characteristics of a track-and-field athlete as well as the specialized skills of a swimmer, with both branches sharing a common base athlete record.



Requirements

Question 1 (2.0 points): Implement the top-level base class `VAN_DONG_VIEN`.

- Encapsulated data members: `ma_vdv` (`string`), `ho_ten` (`string`), and `nam_sinh` (`int`).
- Provide a default constructor, a parameterized constructor, and a virtual destructor (`virtual ~VAN_DONG_VIEN()`).
- Provide the virtual methods `virtual void nhap()`, `virtual void in() const`, and `virtual double tinh_diem_xep_hang() const`.

Question 2 (2.0 points): Static data member and static method.

- Declare `static int dem_vdv` to count the total number of athlete records currently in memory.
- Automatically update `dem_vdv` in the constructors and destructor.
- Define the static variable outside the class and implement `static void in_tong_so_vdv()`.

Question 3 (2.0 points): Overload the comparison operator `operator<`. Overload `bool operator<(const VAN_DONG_VIEN& other) const` to compare the overall ranking scores returned by `tinh_diem_xep_hang()`. Return `true` if the ranking score of the left-hand object is lower than that of the right-hand object.

Question 4 (2.0 points): Design the diamond-inheritance structure.

- **Branch 1:** `VDV_DIEN_KINH` virtually publicly inherits from `VAN_DONG_VIEN` and adds `so_km_chay_tuan` (`double`) and `thanh_tich_chay_s` (`double`).
- **Branch 2:** `VDV_BOI_LOI` virtually publicly inherits from `VAN_DONG_VIEN` and adds `so_km_boi_tuan` (`double`) and `thanh_tich_boi_s` (`double`).
- **Converging class:** `VDV_TRIATHLON` inherits from both branches.
- Initialize `VAN_DONG_VIEN` directly in the member-initializer list of `VDV_TRIATHLON`.
- Add the private data member `so_km_dap_xe_tuan` (`double`).
- Override `nhap()` and `in()`.
- Implement the following ranking-score formula:

Ranking Score = (`so_km_chay_tuan`$\times 1.2$)+(`so_km_boi_tuan`$\times 3.0$)+(`so_km_dap_xe_tuan`$\times 0.5$)

Question 5 (2.0 points): Manage dynamic arrays and sorting in `main()`.

- Call the static method to display the initial object count.
- Dynamically allocate an array of n `VDV_TRIATHLON` athletes.
- Input the data and verify that the number of objects has increased.
- Sort the list in descending order of ranking score using `operator<`.
- Display the sorted list in a properly formatted table.
- Release the memory using `delete[]` and verify that the object count has returned to zero.